

6-DoF Pose Estimation via Learned Surface Embeddings

Master's thesis interview prep — Siemens Foundational Technologies, Garching · Job ID 505049 · framed from the title (posting was removed), SurfEmb-style direction

THE TOPIC — OPENING SUMMARY

- Use **learned dense surface embeddings** to solve **6-DoF pose**: full **position (3)** + **orientation (3)** of a workpiece
- Why: part won't arrive in the same pose each run (bins, shifted tooling, conveyor) → robot must *see* it before acting
- Network maps each surface point into a feature space → sensed points match to points on the known CAD model
- Output = rigid transform (R, t) aligning model → camera/robot frame; aligned cloud is just visual confirmation
- Replaces hand-crafted descriptors (FPFH, SIFT) that fail on **textureless / shiny / symmetric** parts & under occlusion
- Say "from the title I understand the direction to be..." — don't overclaim

CORE THEORY

- **6-DoF** = 3 translation + 3 rotation. Say "position & orientation", not "orientation & rotation"
- **Rotation reps**: matrix · Euler (gimbal lock) · quaternion (double cover) · continuous 6D (best for NN regression)
- **Embeddings > direct regression**: direct assumes one answer (uni-modal) → breaks on symmetry; embeddings learn a *distribution* → symmetry handled implicitly
- **Contrastive loss**: pull true pixel ↔ point pairs together, push others apart. Positives = correspondences under known render pose; negatives = other sampled surface points
- **Corr. → pose**: PnP + RANSAC (RANSAC rejects outlier matches)
- **Coarse vs fine registration**: two stages — one-shot is unreliable
 - **Coarse (global)**: "right ballpark" from *features* — match descriptors on source & model, solve rough (R, t) (e.g. SAC-IA + FPFH). No good start needed; approximate
 - **Fine = ICP**: for each source point find *closest* target point → minimize total distance → repeat to converge. Accurate but only *locally*
 - **ICP weaknesses**: (1) **local minima** — walks downhill, gets stuck; (2) **init-sensitive** — only works if it *starts* close → why coarse runs first
 - **Go-ICP**: guarantees *global* optimum by searching all of **SE(3)** (all rigid transforms) via **branch-and-bound** — split space, bound error per region, discard regions that can't win. Slower → paper adds PSO to speed it up
- **PointNet vs PointNet++**: a point cloud is an *unordered set* → output must not depend on order (**permutation invariance**)
 - **PointNet**: same shared MLP on every point → combine via **max-pool** (symmetric = order-independent). Captures *global* shape, misses *local* detail
 - **PointNet++**: applies PointNet *locally* in stages — **sample** centers (farthest-point) → **group** neighborhoods (multi-scale radii) → extract local features → repeat at larger scales. Builds *local* + *global* hierarchically, like a CNN
- **Sim-to-real gap**: synthetic → real noise/lighting/reflectance; fix w/ domain randomization, PBR rendering, small real fine-tune set

MY PROJECT BRIDGES

- **Embeddings / latent spaces** → my VAE work — a VAE *is* a learned latent rep; embeddings = same idea per-point
- **Sim-to-real** → **Gazebo** sim at DTU + interest in bridging sim & physical experiments
- **Point clouds / registration** → **DTU ROS2** (LiDAR, point clouds, SLAM) + **MTU thesis** (large 3D data, surfaces, deformation fields)
- **Image → 3D geometry** → **stereo table-tennis pipeline** + **Kalman** (real-time 2D → 3D)
- **Industrial / why Siemens** → **Nebumind**: SINUMERIK, Industrial Edge, DH kinematics — machine tending = this pipeline's endpoint **strongest card**
- **Gap framing**: "haven't done pose directly, but I've assembled the building blocks & want to combine them"

NETWORK WORKFLOW

- **Input**: RGB / RGB-D crop of detected object (or point cloud) + known CAD model
- **Output**: rigid pose (R, t), model → robot frame
- **Pipeline**: detect/segment → embed (query=pixels, key=surface pts) → match in embedding space → sample PnP+RANSAC → score → refine (ICP)
- **Point-cloud extra**: background removal, voxel downsample, statistical outlier filter
- **Labels = mostly free**: render CAD in known poses → GT pose + correspondences automatic; no manual symmetry labels

BENCHMARKING & TUNING

- **Metrics**: **ADD / ADD-S** (ADD-S = closest-point, for symmetric) · rotation (geodesic) & translation (Euclidean) error · RMSE
- **BOP metrics**: VSD, MSSD, MSPD → Average Recall
- **Industrial KPIs**: cycle time / latency, pick success rate, false-positive rate
- **Datasets**: BOP suite — name **T-LESS & ITODD** (textureless / industrial)
- **Hyperparams**: embedding dim · contrastive temperature · #negatives · backbone depth · LR / batch / optimizer · augmentation strength · RANSAC iters + inlier threshold · #pose hypotheses · crop resolution

KALMAN FILTER

- Recursive estimator: fuses **noisy measurement** + **model prediction** → smoother estimate over time
- Two steps: **predict** (motion model) → **update** (correct w/ measurement, weighted by uncertainty via Kalman gain)
- Assumes linear + Gaussian; nonlinear → **EKF / UKF**
- **My hook**: used one in the table-tennis robot to predict ball trajectory from noisy stereo

VIDEO + KALMAN → POSE TRACKING

- Per-frame pose is noisy/jittery; KF adds **temporal consistency**
- **State** = pose (+velocity); **measurement** = network's per-frame pose
- **Predict** next-frame pose → **update** with new estimate
- Benefits: smooths jitter, rejects outlier frames, survives brief occlusion / dropped detection
- = **tracking** not single-shot — relevant for moving conveyor / arm-mounted camera
- *Rotations are nonlinear → use EKF/UKF or filter on a proper rotation rep*

STEREO, DEPTH & SCALE

- **Why stereo:** mono has scale ambiguity (small-near ≈ big-far); stereo → depth resolves scale
- **Depth:** match L/R → disparity → depth via baseline + focal length (triangulation)
- **Into pipeline:** (a) depth as extra channel = RGB-D input, or (b) back-project depth+intrinsics → point cloud → 3D-3D registration
- **Preprocessing (yes, before NN):** calibration (intrinsics+extrinsics) + rectification · depth denoise · handle invalid depth (**shiny metal** returns bad depth)
- **Payoff:** depth gives metric scale → (R, t) in real **mm** the robot can act on

TWO-TOWER (QUERY-KEY) MODEL

- **Query tower** — in: image crop (2D pixels); out: **per-pixel embedding**; arch: CNN encoder-decoder (U-Net / ResNet)
- **Key tower** — in: a 3D surface coordinate (CAD); out: embedding of that point in the *same* space; arch: small MLP
- **Link:** both map into one **shared embedding space**, trained jointly w/ contrastive loss so matching pixel/point embeddings land close
- **Inference:** for each pixel embedding → nearest surface-point embedding → dense 2D-3D correspondences → pose
- One tower speaks "2D image", the other "3D surface"; shared space lets them talk
- **"Small MLP"?** MLP = multi-layer perceptron — stacked fully-connected layers + nonlinearity (ReLU). "Small" = few layers/neurons. The key tower only maps (x, y, z) → embedding (simple input), so a light net suffices; the query tower needs a big CNN for the rich image. Asymmetry is intentional
- *Phrase as "in the SurfEmb line of work" — thesis may use a variant*

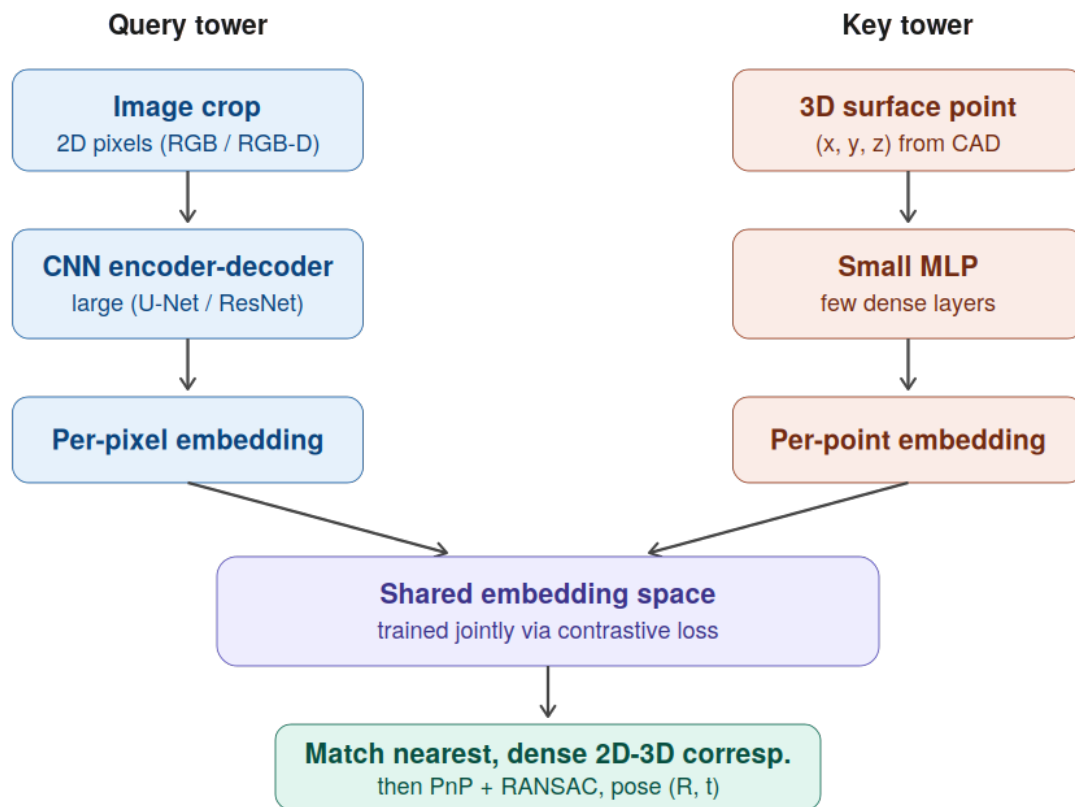
WING / AFP EXTENSION (MY OWN IDEA)

- Rigid 6-DoF = *where is the mold/part*; dense embedding = *which zone of the surface am I on*
- Attach per-zone metadata to canonical model: temperature, roll speed, compaction force
- Correspondence → look up local process rules → adapt path planning in real time
- Optimize: surface-normal accuracy, compaction, defect avoidance (wrinkles, gaps, porosity)
- *Concede: large smooth molds = few features → often CAD-to-scan ICP + metrology; embeddings best for discrete complex parts*

QUESTIONS TO ASK THE SUPERVISOR

- **Topic:** embedding *replaces* registration or *feeds* ICP refinement? · instance-level or category/unseen? · methodological vs pipeline/benchmark split? · main failure mode being beaten?
- **Software:** framework/backbone (PyTorch?) · existing codebase or build from SurfEmb? · synthetic-only (BlenderProc/PBR) or real annotated data? · BOP or Siemens-specific eval?
- **Sensors:** RGB-D / stereo / 3D scanner? · fixed or arm-mounted? · accuracy & cycle-time targets? · real robot to validate on? · how reflective/symmetric are the parts?

TWO-TOWER WORKFLOW & LINKAGE — SCHEMATIC



Read top-down: the two towers stay **separate** until the purple box — that separation *is* the architecture. The **query tower** (heavy CNN) turns the image into per-pixel embeddings; the **key tower** (small MLP) turns each 3D CAD surface point into a per-point embedding. The **linkage** is that both are trained *jointly* to land in **one shared embedding space** via the contrastive loss, so a pixel and its true surface point sit close together. At inference you **match** each pixel embedding to its nearest surface-point embedding → dense 2D-3D correspondences → PnP + RANSAC → pose (R, t) . One tower speaks "2D image", the other "3D surface"; the shared space is the common language that lets them be compared.